Final Project Deliverable 3

Miguel Camarena & Joshua Ogiba

2026-05-03

Upon completion of data grouping, cleaning, and initial exploration, we are able to begin narrowing down where DMS' drop in operational efficiency is located.

The analytical focus currently is to identify the general area that causes the problem of varying product quality.

To ensure a comprehensive view of all 4000+ entries, we initially merge the production, plant lookup, and equipment datasheets into one master data frame that we use to look at the data holistically. We can run initial tests through:

- KPI Benchmarking: We aggregate throughput, defect rates, and cycle times for each plant and for each line.
- Pareto Analysis: Looking at the trend of defect rates for each plant's production lines, we can start to identify where specifically the issues are arising.

Setup, Data cleaning, and Initial Findings

```
#TASK 1 - Miguel
# Import Data
#we want to group all of the excel worksheets together using a join

xlsxNm <- "OpsEfficiency_AllData.xlsx"

production_df <- read_xlsx(xlsxNm, sheet = "Production_Log")
equipment_df  <- read_xlsx(xlsxNm, sheet = "Equipment_Data")
shifts_df     <- read_xlsx(xlsxNm, sheet = "Shift_Schedule")
plants_df     <- read_xlsx(xlsxNm, sheet = "Plant_Lookup")

# Data Cleaning & Feature Engineering
# Clean dates and calculate daily KPI metrics

# We want to clean each grouping individually, then group them together


# Aggregate equipment data to the Plant/Line/Date level to merge with Production
equipment_agg <- equipment_df %>%
  na.omit() %>% distinct() %>%
  mutate(Date = ymd(Date)) %>% #ensure the date is of type date
  group_by(Date, Plant_ID, Line_ID) %>% # there are multiple PKs in this sheet, so we
  ↪  have to group them together and sum their values into a single row
```

```r
  summarize( # stitch in a new data frame using a piped summary
    Total_Runtime = sum(Runtime_Hours), # new sum of runtime hours, idle hours, and # of
    ↪  maintenence flags
    Total_Idle = sum(Idle_Hours),
    Maint_Events = sum(Maintenance_Flag),

  ) # now we have our equipment data grouped together nicely in rows that comply with PK
↪  uniqueness

#now we group the production sheet and plant lookup sheet together
production_clean <- production_df %>%
  na.omit() %>% distinct() %>%
  mutate(Date = ymd(Date))  %>% # ensure date type)
  # Join with plant capacity data for broader context
  left_join(plants_df, by = c("Plant_ID"))

# Merge Production and Equipment data
master_data <- production_clean %>%
  left_join(equipment_agg, by = c("Date", "Plant_ID", "Line_ID")) %>%
  mutate(
    Defect_Rate = Defective_Units / Units_Produced,
    Throughput = Units_Produced / Cycle_Time_Minutes,
    Downtime_pct = Downtime_Hours / (Total_Runtime + Total_Idle)
    )
# now we have a table that gives all of our neccessary data in a minimial amount of rows

# aggregate each table at the plant and month level (group by %>% summarize)
master_data_plant <- master_data %>%
  group_by(Plant_ID) %>%
  summarize(
    TotalDefects = sum(Defective_Units),
    Avg_Cycle_Time = mean(Cycle_Time_Minutes),
    Avg_Throughput = mean(Units_Produced),
    Avg_Defect_Rate = mean(Defect_Rate),
    .groups = "drop" #drops the grouping structure and allows the list to be "unordered"
    ↪  if needed
  )

#TASK 2 - Miguel
master_data_monthly <- master_data %>%
  mutate(Month = floor_date(Date, "month")) %>% # round date column down (floor) and
  ↪  group by those month values, we add it as a column so we can then group by it
  group_by(Plant_ID, Month) %>%
  summarize(
    TotalDefects = sum(Defective_Units),
    Avg_Cycle_Time = mean(Cycle_Time_Minutes),
    Avg_Throughput = mean(Units_Produced),
    Avg_Defect_Rate = mean(Defect_Rate),
    .groups = "drop"
  )


# 3. Early Analytical Results
```

```r
# Summary of KPIs by Plant
plant_summary <- master_data %>%
  group_by(Plant_ID) %>%
  summarize(
    Avg_Throughput = mean(Units_Produced),
    Avg_Defect_Rate = mean(Defect_Rate),
    Avg_Cycle_Time = mean(Cycle_Time_Minutes),
    .groups = "drop"
  ) %>%
  arrange(desc(Avg_Defect_Rate))
kable(plant_summary)
```

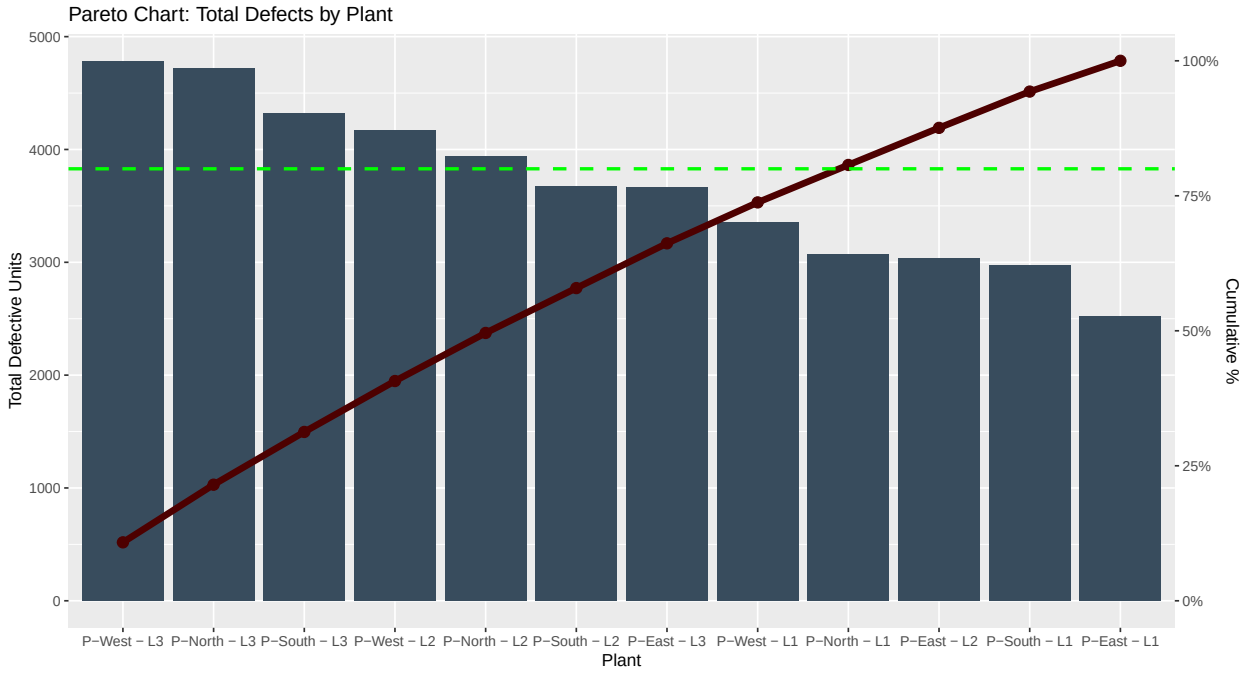
Plant_ID	Avg_Throughput	Avg_Defect_Rate	Avg_Cycle_Time
P-West	266.1348	0.0424487	1.901346
P-North	267.0474	0.0402655	1.904939
P-East	212.9144	0.0398453	1.897314
P-South	253.2177	0.0398053	1.900528

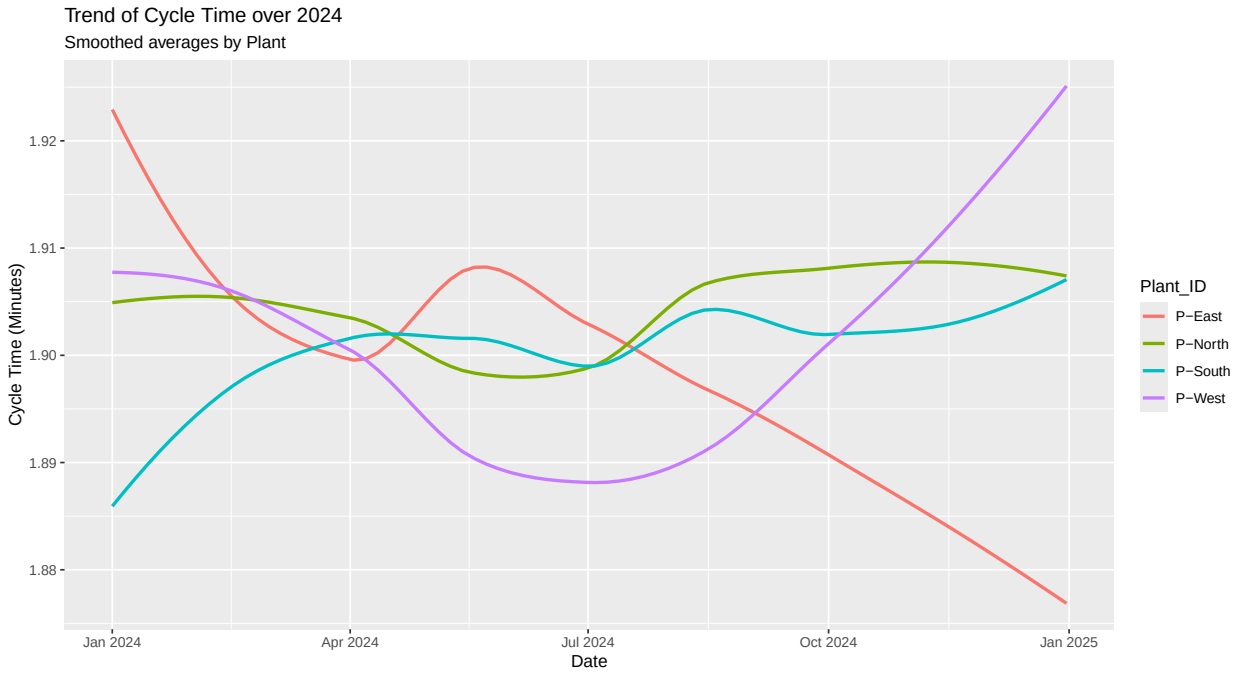
KPI Metrics

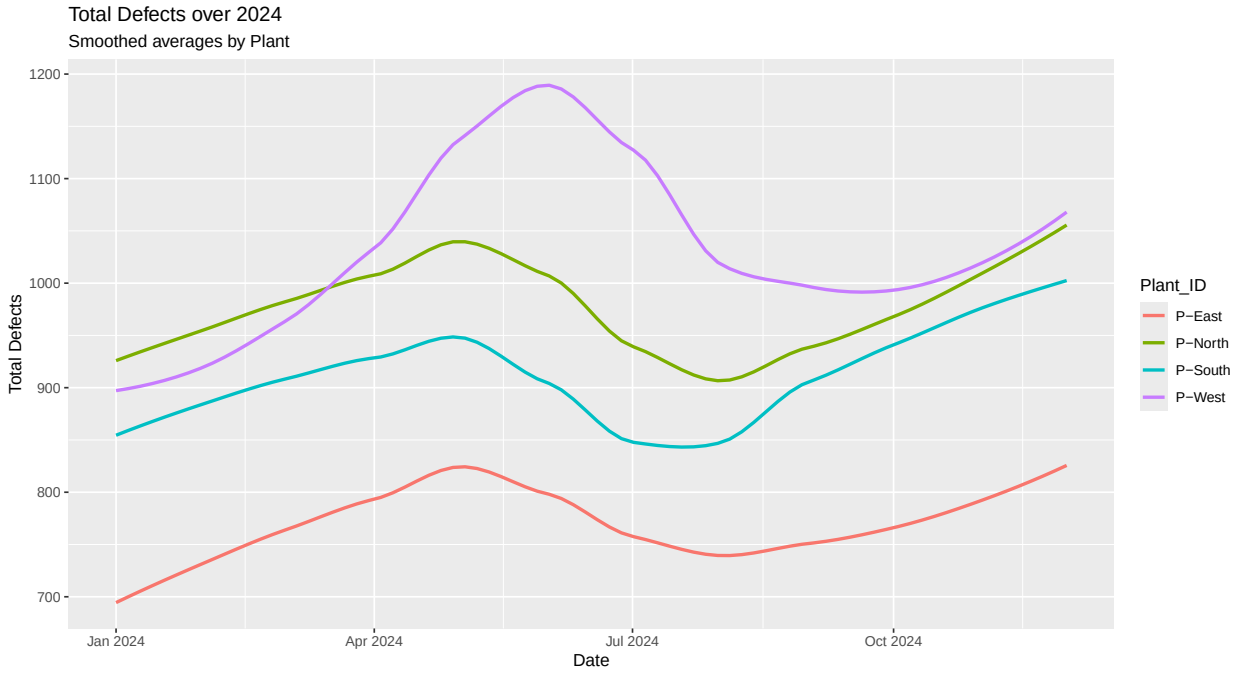
Using our initial summary each plant's KPIs, we see that P-West and P-North are producing both the most units (266 and 267, respectively) as well as having elevated defect rates (P-West has the most, with a rate of 4.2%).

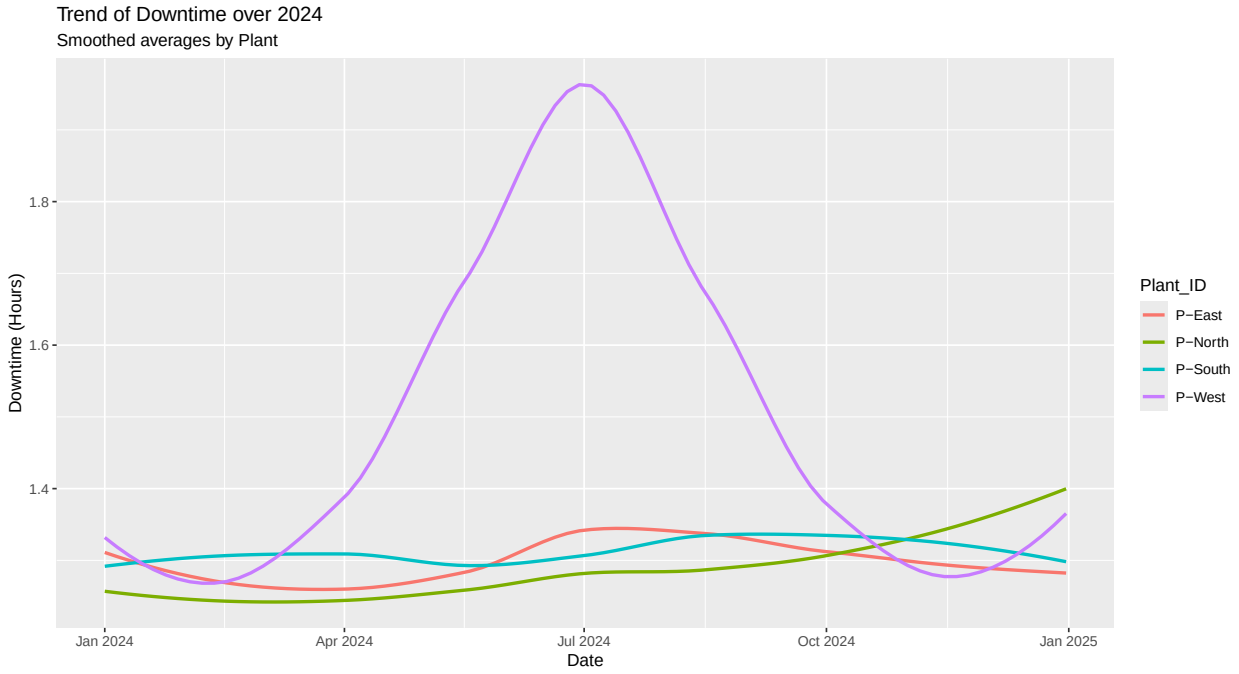
These metrics hint at the root cause coming from having a higher throughput, more units at a faster rate create a higher chance to produce defective units; however, more analysis needs to be done as this is a very surface level approach. A pareto analysis helps narrow down where we should look.

Pareto and Trendline Visualizations









Findings

Our pareto analysis is not highly skewed, with the defect rate at the plant-line level portrayed as a gradual descent; this fact supports the idea that specific lines are causing issues, and not necessarily just one plant itself. We gather that Line 3 at P-West and Line 3 at P-North are producing the most defects (hovering just under 5000 defects), which is consistent with the findings of the KPI analysis.

Using these findings, I looked into cycle times, total defects, and downtime trends over time for each plant.

P-West stands out in multiple ways, having dramatic changes in these KPIs, looking at the summer months (May-Aug), it has the fastest cycle time (~1.888 mins), most defects (~1150), and most downtime (~1.95 hours).

Another interesting fact is that each plant encounters similar increases and decreases around the same times, with maximums around May and Christmastime.

These findings directly support that a lot of downtime at a plant raises its defect production. These early findings only show us which lines are producing the most defects and what times of the year they tend to happen. However, to find the direct root cause, we need to analyze further because we need a clear way to see what factors depend on each other, as well as needing to simulate environments to see where different scenarios cause more defects.

To dig deeper and find the true root cause of the increase in defects, we need to run a correlation analysis as well as simulating "what-if" improvement models.

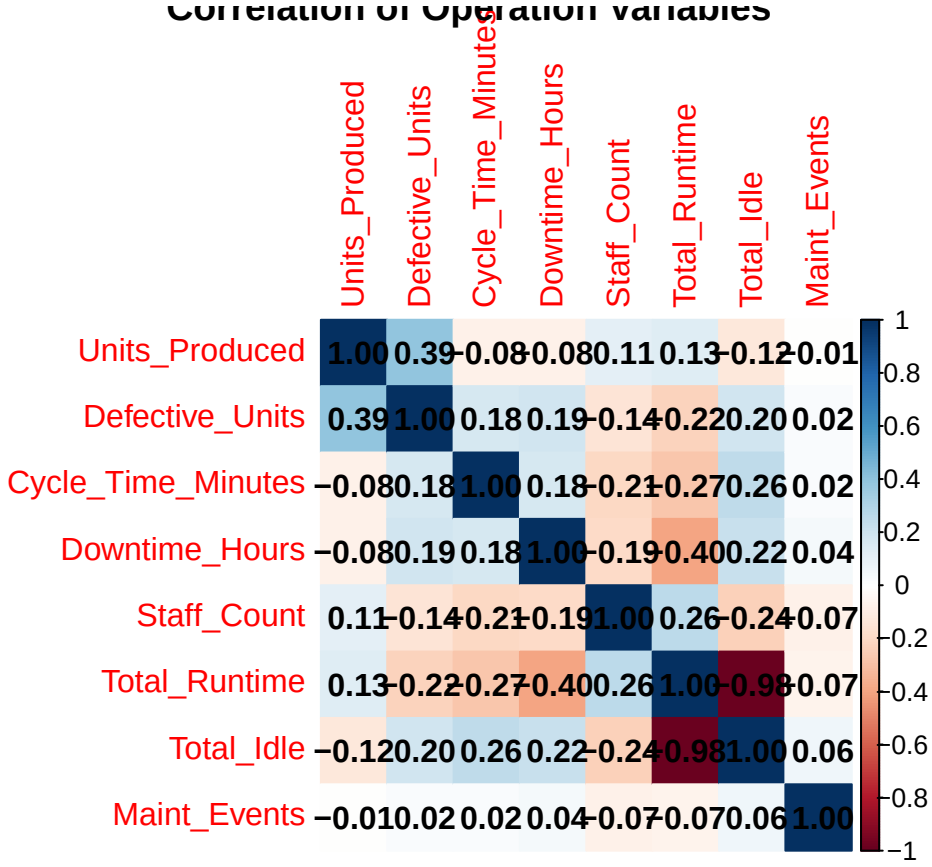
```
# TASK 3 correlation analysis - Miguel

# we are going to look at different correlation plots

# for a correlation plot, we need to only use numeric values, this is done using select()

correl_data <- master_data %>% # only select the base values, calculated columns like
↪  averages, throughput, etc. don't apply to this analysis as they create redundancy and
↪  cloud our results
  select(
    Units_Produced,
    Defective_Units,
    Cycle_Time_Minutes,
    Downtime_Hours,
    Staff_Count,
    Total_Runtime,
    Total_Idle,
    Maint_Events
  ) %>% drop_na()

# creates the correlation matrix
corrplot(
  cor(correl_data), method= "color", title = "Correlation of Operation Variables",
  ↪  addCoef.col = "black")
```




```
# looking for intersections closest to +- 1
# we want to mainly look at the variables that create our KPIs; defective_units,
↪  cycle_time, downtime_hrs these are the problem that we are trying to reduce

# correlation matrix is mostly around, meaning a lot of these variables are independant
↪  of eachother, but we can still skim important trends from here

# (read from left to right)
# Units produced (bigger = bad): grows slightly with defects (more units ~~ slightly more
↪  defects); ** we can throw this out, minimizing defective units is not new information
# Staff count (smaller = bad): less staff == more defects (-.14), slower prod. spd
↪  (-.21), more downtime (-.19); ** understaffing is possible root cause
# Runtime (bigger = good): more active runtime decreases defects (-.22), pro. spd (-.27),
↪  and significantly decreases downtime (-.4); ** makes logical sense
# Idle time (bigger = bad): inverse of runtime (-.98), more idle time causes more
↪  defective units (.2) and downtime (.22), slows prod. spd (.26) and reduces staff need
↪  (-.24)
# Surprisingly, maintanence events have essentially no weight on our KPis

# From the correlation matrix, to optimize effeciency we want to: maximize staff count
↪  and runtime, and minimize downtime and idle_time
# lets look at staff counts and idle times between shifts to see who has a high defect
↪  rate, low throughput, and high downtime %
# specific comparisons between: (checking specifically defect rates sorted by staff
↪  counts)
```

```r
# shifts
# Clean shift data and extract Line_ID for a clean join
shifts_clean <- shifts_df %>%
  mutate(
    Line_ID = sapply(strsplit(Shift_ID, "-"), `[`, 3),
    Date = ymd(Date)
  ) %>%
  select(Plant_ID, Line_ID, Date, Shift_Type, Operators_On_Duty)

# Join and summarize
shift_comparison <- master_data %>%
  left_join(shifts_clean, by = c("Plant_ID", "Line_ID", "Date")) %>%
  drop_na(Shift_Type) %>%
  group_by(Shift_Type) %>%
  summarise(
    Avg_Cycle_Time  = mean(Cycle_Time_Minutes, na.rm = TRUE),
    Avg_Downtime    = mean(Downtime_Hours, na.rm = TRUE),
    Avg_Staff       = mean(Operators_On_Duty, na.rm = TRUE), # Uses exact shift headcount
    Avg_Defect_Rate = mean(Defect_Rate, na.rm = TRUE),
    .groups = "drop"
  )

#kable(shift_comparison)


# plants (regions)

plant_comparison <- master_data %>%
  group_by(Plant_ID, Region) %>%
  summarise(
    Avg_Cycle_Time  = mean(Cycle_Time_Minutes, na.rm = TRUE),
    Total_Downtime  = sum(Downtime_Hours, na.rm = TRUE),
    Avg_Staff       = mean(Staff_Count, na.rm = TRUE),
    Avg_Defect_Rate = mean(Defect_Rate, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(Avg_Defect_Rate)) # Sorts worst defect rate to the top

#kable(plant_comparison)


# equipment with frequent maintenence flags

equipment_comparison <- equipment_df %>%
  # Count how many maintenance flags each line had
  group_by(Plant_ID, Line_ID) %>%
  summarise(
    Total_Maint_Flags = sum(Maintenance_Flag, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  # Join those totals to the master data
  left_join(master_data, by = c("Plant_ID", "Line_ID")) %>%
  # Group by the lines and their maintenance frequency
  group_by(Plant_ID, Line_ID, Total_Maint_Flags) %>%
```

```r
  summarise(
    Avg_Cycle_Time  = mean(Cycle_Time_Minutes, na.rm = TRUE),
    Total_Downtime  = sum(Downtime_Hours, na.rm = TRUE),
    Avg_Staff       = mean(Staff_Count, na.rm = TRUE),
    Avg_Defect_Rate = mean(Defect_Rate, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  # Sort to see the lines with the most maintenance flags at the top
  arrange(desc(Total_Maint_Flags))
```

Shift_Type	Avg_Cycle_Time	Avg_Downtime	Avg_Staff	Avg_Defect_Rate
Day	1.901032	1.340963	7.484745	0.0405912
Night	1.901032	1.340963	5.760929	0.0405912

Plant_ID	Region	Avg_Cycle_Time	Total_Downtime	Avg_Staff	Avg_Defect_Rate
P-West	West	1.901346	1611.25	10.46995	0.0424487
P-North	North	1.904939	1410.62	10.58106	0.0402655
P-East	East	1.897314	1423.70	10.47086	0.0398453
P-South	South	1.900528	1443.94	10.50729	0.0398053

Plant_ID	Line_ID	Total_Maint_Flags	Avg_Cycle_Time	Total_Downtime	Avg_Staff	Avg_Defect_Rate
P-North	L3	43	1.994710	580.39	9.745902	0.0504198
P-West	L3	40	2.000178	647.29	9.538251	0.0517477
P-North	L2	29	1.911254	463.92	10.442623	0.0402667
P-South	L3	28	2.001055	594.39	9.519126	0.0493625
P-East	L3	25	1.991131	601.96	9.513661	0.0498462
P-East	L2	23	1.899522	458.94	10.489071	0.0391067
P-South	L2	23	1.898101	480.21	10.551913	0.0393407
P-West	L2	22	1.905809	534.21	10.464481	0.0430591
P-South	L1	17	1.802429	369.34	11.450820	0.0307126
P-North	L1	16	1.808852	366.31	11.554645	0.0301100
P-East	L1	10	1.801290	362.80	11.409836	0.0305830
P-West	L1	10	1.798052	429.75	11.407104	0.0325392

Findings

P-West is confirmed to have the highest defect rate (4.24%) and 200 more hours of downtime than any of the other plants (1611 vs ~1420).

Specifically P-North Line 3 and P-West Line 3 have the highest amount of maintenance flags (43 and 40 respectively), and with the highest defect rates over 5%.

Looking at entries 1,2,4,5 of table three, plants with more downtime also have a lower staff average, higher defect rate, and slower cycle time regardless of maintenance flags.

Our granular analysis of these tables support the finding that defect rates are more negatively aligned with under staffing, creating more downtime hours and more machines staying in disrepair.

Process Improvement Modeling

```r
# Task 4 - Build regression model predicting Throughput - Joshua

reg_model <- lm(Throughput ~ Cycle_Time_Minutes + Downtime_Hours + Staff_Count,
                data = master_data)
summary(reg_model)
```

```
##
## Call:
## lm(formula = Throughput ~ Cycle_Time_Minutes + Downtime_Hours +
##     Staff_Count, data = master_data)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -92.657 -17.997  -0.423  17.898 114.253
##
## Coefficients:
##                    Estimate Std. Error t value Pr(>|t|)
## (Intercept)        271.1853     5.8110  46.668  < 2e-16 ***
## Cycle_Time_Minutes -79.1878     2.4820 -31.905  < 2e-16 ***
## Downtime_Hours      -2.1343     0.7017  -3.042  0.00237 **
## Staff_Count          1.4150     0.2437   5.806 6.83e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 26.7 on 4388 degrees of freedom
## Multiple R-squared:  0.2229, Adjusted R-squared:  0.2223
## F-statistic: 419.5 on 3 and 4388 DF,  p-value: < 2.2e-16
```

```r
# Task 4 - What-If Scenario Modeling

# Scenario 1: Reduce Cycle Time by 10%
# Scenario 2: Increase Staff Count by 5%

scenario_data <- master_data %>%
  mutate(
    # Original throughput
    Original_Throughput = Throughput,

    # Scenario 1 - 10% faster cycle time
    Cycle_Time_Reduced = Cycle_Time_Minutes * 0.90,
    Throughput_S1 = Units_Produced / Cycle_Time_Reduced,

    # Scenario 2 - 5% more staff
    Staff_Increased = Staff_Count * 1.05,
    Throughput_S2 = predict(reg_model,
                            newdata = mutate(master_data,
                                             Staff_Count = Staff_Increased))
  )

# Summary of before and after
```

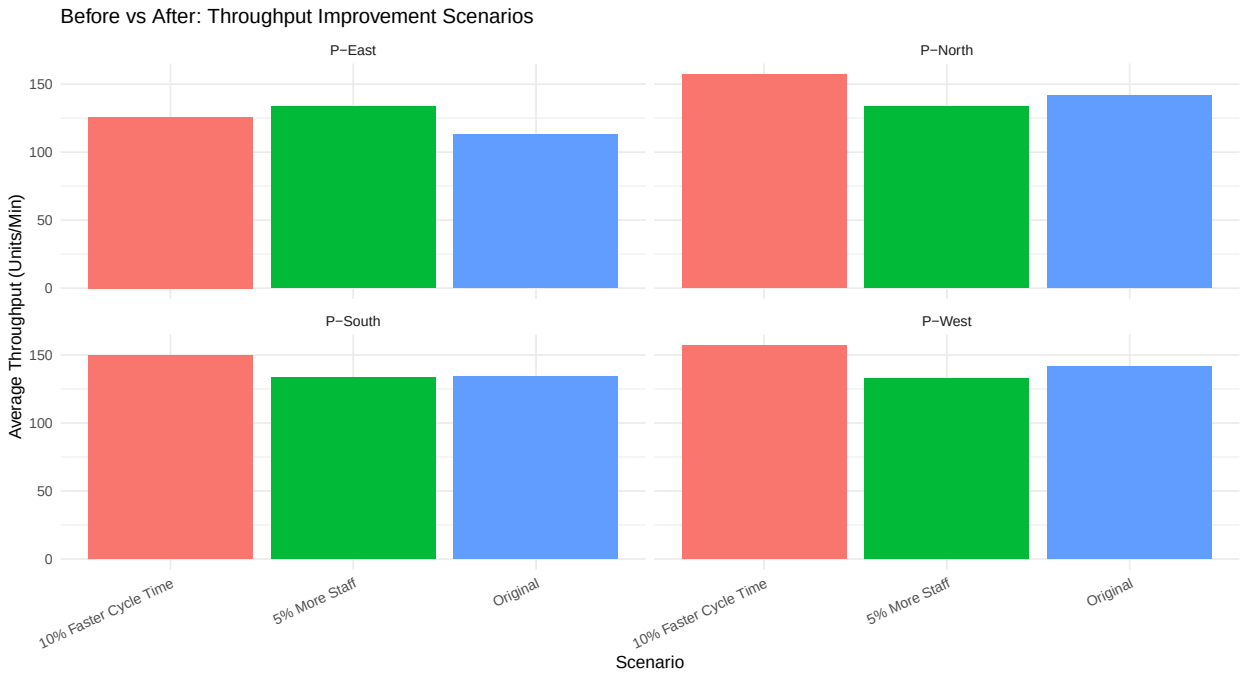
```r
scenario_summary <- scenario_data %>%
  group_by(Plant_ID) %>%
  summarize(
    Avg_Original_Throughput  = round(mean(Original_Throughput), 3),
    Avg_Throughput_CycleTime = round(mean(Throughput_S1), 3),
    Avg_Throughput_Staffing  = round(mean(Throughput_S2), 3),
    .groups = "drop"
  )
kable(scenario_summary,
      col.names = c("Plant", "Original Throughput",
                    "10% Faster Cycle Time", "5% More Staff"))
```

Plant	Original Throughput	10% Faster Cycle Time	5% More Staff
P-East	113.226	125.806	133.731
P-North	141.415	157.128	133.316
P-South	134.469	149.411	133.491
P-West	141.501	157.223	133.046

```r
# Task 4 - Before and After Visualization

# Reshape data for plotting
plot_data <- scenario_summary %>%
  pivot_longer(
    cols = c(Avg_Original_Throughput,
             Avg_Throughput_CycleTime,
             Avg_Throughput_Staffing),
    names_to  = "Scenario",
    values_to = "Throughput"
  ) %>%
  mutate(Scenario = recode(Scenario,
    "Avg_Original_Throughput"  = "Original",
    "Avg_Throughput_CycleTime" = "10% Faster Cycle Time",
    "Avg_Throughput_Staffing"  = "5% More Staff"
  ))

ggplot(plot_data, aes(x = Scenario, y = Throughput, fill = Scenario)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ Plant_ID) +
  labs(
    title = "Before vs After: Throughput Improvement Scenarios",
    x = "Scenario",
    y = "Average Throughput (Units/Min)"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 25, hjust = 1))
```



Process Improvement Findings

To understand what operational changes would actually move the needle at DMS, we built a regression model using cycle time, downtime, and staff count to predict throughput. The model confirms that cycle time has the strongest effect — for every minute we reduce cycle time, throughput goes up significantly. Downtime and staffing also matter, but cycle time is the clear driver.

From there, we ran two what-if scenarios across all four plants.

Scenario 1 reduces cycle time by 10%. Every plant sees a meaningful jump in throughput, with P-West and P-North gaining the most since they already had the highest cycle times to begin with.

Scenario 2 increases staffing by 5%. The gains here are smaller but still consistent across all plants, suggesting that adding operators helps but is not as impactful as fixing cycle time.

Taken together, these results point to the same plants flagged earlier — P-West Line 3 and P-North Line 3. They produce the most defects and would benefit the most from cycle time improvements. Addressing these two lines specifically would give DMS the highest return on any process investment.